MONOLITHIC ARCHITECTURE APPLICATION

Project Documentation

Step-by-Step Implementation, API Documentation, Testing and Troubleshooting

Technology: Java • Spring Boot • Spring Data JPA • Maven • REST API
Architecture: Monolithic Application

1. Introduction

This project demonstrates the conversion of a microservices-based application into a monolithic Spring Boot application. In the monolithic version, User, Product and Order functionalities are maintained inside a single Spring Boot application and are deployed as one application.

2. Objective

- Understand the difference between microservices and monolithic architecture.
- Combine User, Product and Order modules into one Spring Boot project.
- Implement REST APIs for CRUD operations.
- Use Spring Data JPA repositories for database persistence.
- Create service classes for business logic and controller classes for REST endpoints.
- Test the application using cURL or a REST client.
- Document the complete implementation and testing process.

3. Architecture

In the monolithic architecture, all modules run within the same Spring Boot application. The controller receives an HTTP request, the service performs business logic, and the repository communicates with the database.

Logical flow:

Client → Controller → Service → Repository → Database

Modules in this project:

• User module
• Product module
• Order module

4. Project Structure

```
demo/
├── pom.xml
└── src/
    └── main/
        └── java/
            └── com/example/demo/
                ├── DemoApplication.java
                ├── user/
                │   ├── controller/UserController.java
                │   ├── entity/User.java
                │   ├── repository/UserRepository.java
                │   └── service/UserService.java
                ├── product/
                │   ├── ProductController.java
```

```
│   ├── entity/Product.java
│   ├── repository/ProductRepository.java
│   └── service/ProductService.java
└── order/
    ├── OrderController.java
    ├── entity/Order.java
    ├── repository/OrderRepository.java
    └── service/OrderService.java
```

5. Technologies Used

Technology	Purpose
Java	Programming language
Spring Boot	Application framework
Spring Web	REST API development
Spring Data JPA	Database access and CRUD operations
Maven	Build and dependency management
REST API	Communication between client and application
VS Code	Development environment
Database	Persistent storage for application entities

6. Step-by-Step Implementation

Step 1 – Create the Spring Boot project

Create/open the Spring Boot project named demo. Verify that pom.xml is present in the demo folder.

Step 2 – Configure dependencies

The pom.xml should contain Spring Boot Web, Spring Data JPA and the database driver required by the project.

Step 3 – Create packages

Create separate packages for user, product and order. Within each module create controller, entity, repository and service classes as required.

Step 4 – Create the User entity

Create User.java with the fields used by the application, such as id, name and email. Mark the class with @Entity and the primary key with @Id and @GeneratedValue.

Step 5 – Create UserRepository

Create UserRepository extending JpaRepository<User, Long>. This provides findAll, findById, save, existsById and deleteById operations.

Step 6 – Create UserService

Keep business logic in UserService. Implement getAllUsers, getUserById, createUser, updateUser and deleteUser. For update, first find the existing user, modify its fields, and save it. For delete, check that the user exists before deleting.

Step 7 – Create UserController

Expose REST endpoints such as GET /users, GET /users/{id}, POST /users, PUT /users/{id} and DELETE /users/{id}.

Step 8 – Implement Product module

Create Product entity, repository, service and controller using the same layered pattern. Implement the required Product CRUD APIs.

Step 9 – Implement Order module

Create Order entity, repository, service and controller. Implement the required Order operations.

Step 10 – Run the application

Open the terminal in the folder containing pom.xml. For this project, the Maven project is inside the demo folder. Run cd demo and then mvn spring-boot:run.

Step 11 – Verify the server

Confirm from the Spring Boot console that the application starts successfully and note the configured server port.

Step 12 – Test the APIs

Test GET, POST, PUT and DELETE operations using cURL, Postman or another REST client.

Step 13 – Validate database operations

Verify that create, update and delete requests are reflected in the database and that GET requests return the expected records.

7. User API Documentation

Method	Endpoint	Purpose	Expected Result
GET	/users	Get all users	Returns list of users
GET	/users/{id}	Get one user	Returns user or 404 if not found
POST	/users	Create user	Creates and returns user
PUT	/users/{id}	Update user	Updates existing user
DELETE	/users/{id}	Delete user	Deletes existing user

8. UserService.java – Reference Implementation

```
package com.example.demo.user.service;

import java.util.List;
import org.springframework.http.HttpStatus;
import org.springframework.stereotype.Service;
import org.springframework.web.server.ResponseStatusException;
import com.example.demo.user.entity.User;
import com.example.demo.user.repository.UserRepository;

@Service
public class UserService {
```

```
    private final UserRepository userRepository;

    public UserService(UserRepository userRepository) {
        this.userRepository = userRepository;
    }

    public List<User> getAllUsers() {
        return userRepository.findAll();
    }

    public User getUserById(Long id) {
        return userRepository.findById(id)
                .orElseThrow(() -> new ResponseStatusException(
                        HttpStatus.NOT_FOUND,
                        "User not found with id: " + id));
    }

    public User createUser(User user) {
        return userRepository.save(user);
    }

    public User updateUser(Long id, User user) {
        User existingUser = userRepository.findById(id)
                .orElseThrow(() -> new ResponseStatusException(
                        HttpStatus.NOT_FOUND,
                        "User not found with id: " + id));

        existingUser.setName(user.getName());
        existingUser.setEmail(user.getEmail());

        return userRepository.save(existingUser);
    }

    public void deleteUser(Long id) {
        if (!userRepository.existsById(id)) {
            throw new ResponseStatusException(
                    HttpStatus.NOT_FOUND,
                    "User not found with id: " + id);
        }
        userRepository.deleteById(id);
    }
}
```

9. API Testing Commands

Get all users

```
curl http://localhost:8081/users
```

Get user by ID

```
curl http://localhost:8081/users/1
```

Create user

```
curl -X POST http://localhost:8081/users -H "Content-Type: application/json" -d
"{\"name\":\"Sai\",\"email\":\"sai@gmail.com\"}"
```

Update user

```
curl -X PUT http://localhost:8081/users/1 -H "Content-Type: application/json" -d "{\"name\":\"Sai
Kumar\",\"email\":\"saikumar@gmail.com\"}"
```

Delete user

```
curl -X DELETE http://localhost:8081/users/1
```

10. Testing Checklist

- Application starts without compilation errors.
- GET /users returns the existing users.
- GET /users/{id} returns the requested user.
- GET /users/{invalidId} returns HTTP 404.
- POST /users creates a new user.
- PUT /users/{id} updates an existing user.
- PUT /users/{invalidId} returns HTTP 404.
- DELETE /users/{id} deletes an existing user.
- DELETE /users/{invalidId} returns HTTP 404.
- Product APIs are tested.
- Order APIs are tested.
- Database records are verified after CRUD operations.

11. Common Errors and Solutions

Error	Solution
No plugin found for prefix 'spring-boot'	Run Maven from the folder containing pom.xml. In this project: cd demo, then mvn spring-boot:run.
HTTP 500 Internal Server Error	Check the Spring Boot terminal stack trace. Verify entity fields, repository methods, database configuration and service logic.
Port 8081 already in use	Find the process using the port or stop the previous application instance before starting again.
404 User not found	Confirm the ID exists. The service should return 404 when findById does not find a record.
Compilation error in updateUser	Make sure the existing user is retrieved before calling setName/setEmail and save.
DELETE behaves incorrectly	Use if (!userRepository.existsById(id)) before deleteById(id).

12. Microservices vs Monolithic Architecture

Aspect	Microservices	Monolithic
Deployment	Multiple services	Single application

Code organization	Services are separated	Modules are inside one application
Communication	Often service-to-service HTTP/messages	Usually direct method/service calls
Scaling	Individual services can be scaled	Application is generally scaled as a unit
Complexity	Higher operational complexity	Simpler to run initially
Database	Can be separated per service	Commonly shared application database

13. Advantages of the Monolithic Version

- Simple deployment because there is one application.
- Easy local development and testing.
- Modules can communicate through normal Java method calls.
- Lower infrastructure and operational overhead for a small project.
- All functionality can be managed from one Spring Boot application.

14. Limitations

- A change in one module may require redeployment of the whole application.
- Independent scaling of individual modules is more difficult.
- As the codebase grows, maintaining module boundaries becomes important.
- A failure in the application can affect all modules.

15. Final Deliverables

- Complete Spring Boot monolithic source code.
- User, Product and Order modules.
- Entities, repositories, services and controllers.
- pom.xml with required dependencies.
- REST API test results.
- Database verification.
- Project documentation.
- Screenshots of successful application startup and API testing, if required by the assignment.

16. Conclusion

The project successfully demonstrates how User, Product and Order functionality can be combined into a single Spring Boot monolithic application. The layered design separates REST controllers, business services, persistence repositories and entities while keeping all modules within one deployable application. CRUD APIs can be tested independently and the application can be run using Maven from the project directory containing pom.xml.